Lecture 11

近似算法

11.1 基本思想与概念

上一讲讨论了 NP 完全性，引入了一类目前还不知道怎样在多项式时间内求解的问题。对于相应的最优化问题，我们通常有如下三个期望：

1. 算法要能找到确切的最优解（optimality）；
2. 算法能高效地运行，这里以多项式时间为标准（efficiency）；
3. 算法适用于这个问题的所有合法实例（all instances）。

除非 P = NP，否则对于 NP 困难的优化问题，不能同时做到这三点。如果舍弃第二个期望，可以用回溯等方法求精确解，这对输入不大的情况是可以接受的；如果舍弃第三个期望，可以寻找容易求解的特例；如果舍弃第一个期望，但能保证高效找到的可行解和最优解相差不大，就得到了本讲要讨论的近似算法。

近似算法是值得一个长学期一门大课来吸收其背后深刻内涵的主题，因此本讲仅仅是介绍几个基本的近似算法的例子，从中体会近似算法的基本设计思路。在正式开始对算法的讨论前，我们首先定义近似算法的一些基本范式和概念。

11.1.1 近似比

假设有一类最优化问题 $\mathcal{I}$，其中的一个具体实例记为 I，A 为一个多项式时间算法。定义 $A(I)$ 为算法 A 在实例 I 上得到的可行解对应的目标值，$OPT(I)$ 为实例 I 的最优解对应的目标值。例如，背包问题就是一类最优化问题，当背包问题的容量、质量和价值等参数给定的时候即为一个最优化问题的实例。算法返回一组总重量不超过容量的物品，这组物品就是可行解。下面我们给出近似算法和近似比的正式定义。

定义 11.1. 如果 $\mathcal{I}$ 是最小化问题，若存在 $r \geqslant 1$，对任意的 I 都有 $A(I) \leqslant r \cdot OPT(I)$，那么称 A 为该问题的 r-近似算法，即对于任何可能的问题实例，A 给出的解都不会比最优解的 r 倍更大。我们特别关心其中可以取到的最小 r，称

$$\rho = \inf\{r : A(I) \leqslant r \cdot OPT(I), \forall I\}$$

为**近似比（approximation ratio）**，即算法 A 最紧的近似界。它可以等价定义为：

$$\rho = \sup_I \frac{A(I)}{OPT(I)},$$

反之，如果是最大化问题，那么上式应该改为

$$\rho = \sup_I \frac{OPT(I)}{A(I)},$$

将两者合并起来，可以统一写作

$$\rho = \sup_I \left\{ \frac{OPT(I)}{A(I)}, \frac{A(I)}{OPT(I)} \right\}.$$

注意上面只讨论了近似比为常数的情况，事实上有时候近似比也可能与输入规模有关，例如接下来会讨论的调度问题，但大部分情况下的结果都是常数的，因此一般而言定义都是常数。

以最小化问题为例，要证明某个算法的近似比为 r，通常分为两步：

1. 首先寻找一个 $r \geqslant 1$，对于任何实例 I，都有 $A(I) \leqslant r \cdot OPT(I)$，例如可以首先找到 $OPT(I)$ 的一个下界 $LB(I) \leqslant OPT(I)$，然后证明 $A(I) \leqslant r \cdot LB(I)$ 即可；
2. 接下来证明 r 是不可改进的，即给出达到 r 的实例，或者证明对任意的 $\varepsilon > 0$，都存在一个实例 I_ε，使得 $A(I_\varepsilon) \geqslant (r - \varepsilon) \cdot OPT(I_\varepsilon)$。

11.1.2 近似方案

自然地，我们希望近似比尽可能接近 1。事实上，对于某些问题，可以根据所要求的近似比调整相应的算法，这就引出了近似方案。用 $\varepsilon > 0$ 表示近似比相对于 1 的偏差，记 $A_\varepsilon(I)$ 为算法返回的可行解的目标值，近似方案要求对每个 $\varepsilon > 0$ 和每个合法实例 I，都有

$$\begin{cases} A_\varepsilon(I) \leqslant (1+\varepsilon)OPT(I), & \text{最小化问题,} \\ OPT(I) \leqslant (1+\varepsilon)A_\varepsilon(I), & \text{最大化问题.} \end{cases}$$

例如，最大化问题中取 $\varepsilon = 0.1$，就要求返回值至少为 $OPT/1.1$。ε 越小，要求的解越接近最优解，但算法可能需要更长时间。因此，讨论近似方案时，除了误差保证，还要看时间复杂度如何随 ε 变化。以下用 n 表示实例的二进制编码长度。

定义 11.2. 如果一个近似方案对每个固定的 $\varepsilon > 0$ 都能在关于 n 的多项式时间内运行，就称为**多项式时间近似方案**（polynomial-time approximation scheme，PTAS）。

这里的关键是固定 ε。例如，运行时间为 $O(n^{1/\varepsilon})$ 的方案满足 PTAS 的要求，因为取 $\varepsilon = 0.1$ 时是 $O(n^{10})$，取 $\varepsilon = 0.01$ 时是 $O(n^{100})$，两者都是多项式时间。但这样的算法即使满足理论要求，也可能因为指数太大而无法实际使用。为了进一步限制提高精度所增加的时间，有下面两种更强的要求。

定义 11.3.
1. **高效多项式时间近似方案**(efficient PTAS,EPTAS)要求运行时间为 $O(f(1/\varepsilon)n^c)$，其中 c 是与 ε 无关的常数，f 是可计算函数。
2. **完全多项式时间近似方案**（fully polynomial-time approximation scheme，FPTAS）进一步要求时间关于 n 和 $1/\varepsilon$ 都是多项式。

例如，$O(2^{1/\varepsilon}n^3)$ 符合 EPTAS 定义。无论精度如何变化，n 的指数始终为 3，增加的时间都体现

在前面的因子中；但不符合 FPTAS 定义，因为对 $1/\varepsilon$ 的依赖是指数级别的。$O(n^3/\varepsilon)$ 符合 FPTAS 定义。根据定义也可以看出，每个 FPTAS 都是 EPTAS，每个 EPTAS 又都是 PTAS。

若把多项式时间的要求适当放宽，还可以定义下面的近似方案。

定义 11.4. 对于每个固定的 $\varepsilon > 0$，如果近似方案的时间为 $n^{(\log n)^{O(1)}}$，就称为**准多项式时间近似方案**（quasi-polynomial-time approximation scheme，QPTAS），其中隐藏的常数 $O(1)$ 可以依赖于 ε。

例如，$n^{\log n} = 2^{(\log n)^2}$ 是准多项式时间。它比任何固定次幂 n^c 增长得快，又比 2^n 增长得慢。根据定义可知，每个 PTAS 都是 QPTAS。

我们也可以对优化问题作如下分类。

定义 11.5.
- NPO：实例是否合法可以在多项式时间内判断，可行解的编码长度有关于 n 的多项式上界，且给定一个候选解，可以在多项式时间内判断其可行性并计算目标值。
- PO：NPO 中能够在多项式时间内求出最优解的问题。
- APX：NPO 中存在常数近似比的多项式时间算法的问题。

此外，所有存在 PTAS 的优化问题组成的问题类也记作 PTAS，EPTAS 和 FPTAS 同理。

于是有

$$\mathsf{PO} \subseteq \mathsf{FPTAS} \subseteq \mathsf{EPTAS} \subseteq \mathsf{PTAS} \subseteq \mathsf{APX} \subseteq \mathsf{NPO}.$$

由于精确算法可以满足任意精度要求，故 $\mathsf{PO} \subseteq \mathsf{FPTAS}$。而对于 PTAS，只要固定 $\varepsilon = 1$，就得到多项式时间的 2-近似算法，所以 $\mathsf{PTAS} \subseteq \mathsf{APX}$。其余包含关系直接来自定义。

若 APX 中每个问题都能通过保持近似精度的多项式时间归约转化为问题 Q，就称 Q 为 APX 困难问题。这里要求归约能将 Q 的 PTAS 转化为原问题的 PTAS，因此，若 Q 有 PTAS，则 APX 中所有问题都有 PTAS。文献中经常使用 APX 困难来说明近似求解的困难程度。

11.1.3 加性误差与渐近近似比

前面的近似比控制算法结果与最优值的比例，但有些场合更关心两者相差多少，就可以用加性误差表达。

定义 11.6. 若多项式时间算法 A 对每个合法实例都返回可行解，并满足

$$|A(I) - OPT(I)| \leqslant K,$$

其中 $K \geqslant 0$ 是与实例无关的常数，就称算法具有固定加性误差保证。

这一要求有时很强。以图着色为例，记图 G 所需的最少颜色数为 $\chi(G)$。假设存在多项式时间算法，总能用至多 $\chi(G) + K$ 种颜色完成着色。取整数 $q > K$，复制 q 份 G，再把 G 的不同副本之间的所有顶点两两连边，得到图 H。不同副本必须使用不同的颜色，因此 $\chi(H) = q\chi(G)$。算法给 H 着色时，若用了 a 种颜色，就有

$$q\chi(G) \leqslant a \leqslant q\chi(G) + K < q(\chi(G) + 1).$$

两边除以 q 后向下取整，便得到 $\lfloor a/q \rfloor = \chi(G)$。由于 q 是常数，构造 H 和运行算法都只需多项式时间，这就能精确求解一般图着色。因此，除非 P = NP，一般图着色不存在这样的固定加性误差算法。

分析最小化问题时，有时只能证明算法所得的目标值不超过最优值的某个倍数再加上一个常数，这时可以用渐近近似比来描述算法的保证。

> **定义 11.7** (绝对近似比与渐近近似比). 设优化问题的目标值非负，算法 A 总能返回可行解，$A(I)$ 表示所得解的目标值，$OPT(I)$ 表示最优值。对于最小化问题，若要求每个实例都满足 $A(I) \leqslant rOPT(I)$，所有可取的 r 的下确界称为算法的**绝对近似比**，即
>
> $$R_A = \inf\{r \geqslant 1 :\ A(I) \leqslant rOPT(I) \text{ 对所有} I \text{ 成立}\}.$$
>
> 这就是前面定义的近似比。若允许多出一个与实例无关的常数 K，只要求 $A(I) \leqslant \alpha OPT(I) + K$，所有可取的 α 的下确界称为算法的**渐近近似比**，即
>
> $$R_A^{\infty} = \inf\{\alpha \geqslant 1 :\ \text{存在常数} K \geqslant 0,\ A(I) \leqslant \alpha OPT(I) + K \text{ 对所有} I \text{ 成立}\}.$$
>
> 选定算法和 α 后，必须能找到同一个 K，使所有实例都满足这一不等式。
> 对于最大化问题，分别将两个定义中的不等式改为 $OPT(I) \leqslant rA(I)$ 和 $OPT(I) \leqslant \alpha A(I) + K$。

对于最小化问题，当 $OPT(I) > 0$ 时，将不等式两边除以 $OPT(I)$，得到

$$\frac{A(I)}{OPT(I)} \leqslant \alpha + \frac{K}{OPT(I)}.$$

随着最优值增大，$K/OPT(I)$ 趋于零，右边就越来越接近 α。但在最优值较小的实例上，这一项可能仍然很大，因此渐近近似比可以小于绝对近似比。

11.2 最小化工时调度问题

第一个具体的例子是之前贪心算法中调度问题的一个变种（例 9.9）。假设有 n 个作业需要分配给 $m \geqslant 1$ 台相同的机器，第 i 个作业需要运行 $l_i > 0$ 的时间。每个作业只能交给一台机器完整执行，同一台机器上的作业依次进行。我们希望安排好这些作业，让所有机器尽早完成工作。因此，要最小化的是最后一台机器完成的时刻，称为工时（makespan）。

若只按总工作量平分，每台机器应分得 $\sum_i l_i/m$。但作业不能随意切开，所以均分只是一个下界，未必能实现。另外，最长的单个作业也必须完整执行。因此

$$OPT \geqslant \max\left\{\max_i l_i,\ \frac{1}{m}\sum_i l_i\right\}. \tag{11.1}$$

11.2.1 一个精确的动态规划

在讨论近似解之前，先看一个动态规划。把场景稍作推广：只有两台机器 A 和 B，但两台机器不同，第 i 个作业交给 A 需要正整数时间 a_i，交给 B 需要正整数时间 b_i。

记 $F[k, x]$ 为分配前 k 个作业时，在 A 的总运行时间恰好为 x 的条件下，B 所需的最短运行时间。第 k 个作业只有两种分配方法。如果交给 A，那么前 $k-1$ 个作业分给 A 的时间必须是 $x - a_k$，B 的

时间仍为 $F[k-1, x-a_k]$；如果交给 B，那么前面的 A 时间仍须为 x，B 的时间则增加 b_k。取两者中较小的一个，得到

$$F[k,x] = \min\{F[k-1, x-a_k],\ F[k-1,x] + b_k\}.$$

若出现负下标（例如 $x - a_k < 0$），则按 $+\infty$ 处理。初始状态为 $F[0,0] = 0$，$F[0,x] = +\infty (x > 0)$。最终需要比较两台机器中较大的负载，因此取 $\min_x \max\{x, F[n,x]\}$。记 $W = \sum_i a_i$，则 x 的范围为 $[0, W]$。因此该算法时间复杂度为 $O(nW)$。

这里又遇到了背包问题中的困难：W 是输入的数值之和，不是其编码长度，因此这是伪多项式算法。事实上，即使 A 和 B 是同样的机器（即所有作业在其上运行时间一致），精确求解也已经困难。

定理 11.8. 给定正整数长度的作业、两台相同的机器及阈值 T，判断是否存在工时不超过 T 的分配，是 NP 完全问题。

证明. 由于分配方案可以多项式时间验证是否工时不超过 T，所以问题属于 NP。下面证明 NP 完全性，我们从划分问题归约。划分问题是 Karp 的 21 个 NP 完全问题之一，该问题表述为：给定若干个正整数，问可否将其划分成和相等的两个部分；换言之，给定 $c_1, \ldots, c_n \in \mathbb{Z}^+$，划分问题问是否存在一个 $S \subseteq \{1, \ldots, n\}$ 使得 $\sum_{i\in S} c_i = \sum_{i\notin S} c_i$。为了将划分问题归约到最小化工时调度问题，构造一个最小化工时调度问题的实例。设输入的作业长度为 $2c_1, \ldots, 2c_n$，$T = \sum_i c_i$。显然这些作业能用两台机器在 T 时间内完成当且仅当两台机器上安排的任务时长一致，这也等价于划分问题存在解。由此实现了从划分问题到最小化工时调度问题的多项式归约，因此定理中的判定问题是 NP 完全的，这也表明最小化工时调度问题是 NP 困难的。 □

11.2.2 Graham 列表调度算法

在证明了困难性后，就可以名正言顺地讨论近似算法了——除非你有自信解决 P = NP。最简单的设计近似算法的思路就是用一些看起来很有道理但不能在所有解上返回最优解的贪心算法。在这里的目标显然是 m 个机器上尽可能做到负载均衡，所以自然地，对每个作业进行调度的时候，我们选择当前工作量最小的机器进行调度。这称为 Graham 列表调度（list scheduling）算法。

定理 11.9. Graham 列表调度算法近似比为 $2 - 1/m$。

需要说明的是，本章的近似算法的时间复杂度分析都非常简单，所以都不再赘述。下面就来证明这一结论。在证明近似比的时候，需要做的就是利用贪心算法的特点。

证明. 设 i 表示算法最后得到的具有最大负载的那台机器，设其工时为 M，j 是分配给这个机器的最后一个作业。我们知道算法每次找到的机器的负载都是当时最小的，现在退回到选择 j 之前，则当时全体机器的负载之和为 $\sum_{k=1}^{j-1} l_k$，因此当时最小负载机器的负载小于等于 $\frac{1}{m}\sum_{k=1}^{j-1} l_k$。因此有

$$M \leqslant l_j + \frac{1}{m}\sum_{k=1}^{j-1} l_k \leqslant l_j + \frac{1}{m}\sum_{k\neq j} l_k,$$

将上式变形有

$$M \leqslant \left(1 - \frac{1}{m}\right) l_j + \frac{1}{m}\sum_{k=1}^{n} l_k. \tag{11.2}$$

我们知道 $l_j \leqslant M^*$（其中 M^* 是最优解的工时）且 $\frac{1}{m}\sum_{k=1}^{n} l_k \leqslant M^*$，代入上面的不等式有 $M \leqslant \left(1-\frac{1}{m}\right)M^* + M^* = \left(2-\frac{1}{m}\right)M^*$。

然后作为近似比一定要是下界，即要对任意的 m 找到一个例子是满足这一近似比的。考虑 m 台机器，$m(m-1)$ 个长度为 1 的作业和 1 个长度为 m 的作业作为输入，那么显然最优解是 m，但用 Graham 列表调度算法解为 $2m-1$，恰好符合上述近似比。因此贪心算法的近似比为 $2-1/m$。 □

11.2.3 LPT 算法

上面近似比紧的例子把最长的作业留在了最后。如果先把长作业安排好，便可以避免这个例子中的问题。因此，在全部作业预先给定时，可以先按长度从大到小排序，再做 Graham 列表调度。这就是 LPT（longest processing time first）算法。

要改进式 (11.2)，需要控制最终负载最大机器上的最后一个作业 j 的长度。如果 l_j 很小，代入该式就能得到更好的上界；如果 l_j 很大，排在它前面的作业也都很大，而下面的引理说明这时 LPT 已经最优。

> **引理 11.10.** 若存在工时不超过 T 的分配，且每个作业长度都大于 $T/3$，则 LPT 的工时也不超过 T。

证明. 任何工时不超过 T 的方案，每台机器最多放两个作业。若作业数不超过 m，则每个作业各放入一台即可。以下设作业数大于 m。

先说明可以把最大的 m 个作业分别放在不同机器上，同时保持工时不超过 T。从任意一个工时不超过 T 的方案出发，这 m 个作业要分给 m 台机器，如果其中一台分到了两个，就必有另一台一个也没分到。设前一台上的两个作业长度为 $x \geqslant y$，后一台上的作业长度为 $u \geqslant v$；若后一台只有一个作业，就令 $v=0$，若它是空的，就令 $u=v=0$。由于 x, y 属于最大的 m 个作业，而 u, v 不属于它们，有 $y \geqslant u \geqslant v$。把两台机器上的作业改为 (x, v) 和 (y, u)，两台的负载都不超过原来的 $x+y \leqslant T$，而这两个大作业就分开了。每次遇到两个大作业在同一台机器上的情况，都可以这样调整，直到最大的 m 个作业各占一台机器。

接着考虑剩下的作业，每台机器至多再接收一个作业。为统一讨论，没有第二个作业的机器可看作接收了一个长度为 0 的作业。我们希望把较长的剩余作业交给当前负载较小的机器。这样安排不会比反过来更差，因为当两台机器的已有负载为 $\alpha \leqslant \beta$，两个待分配作业的长度为 $q \geqslant r$ 时，有 $\max\{\alpha+q, \beta+r\} \leqslant \beta+q \leqslant \max\{\alpha+r, \beta+q\}$。因此，只要较长的作业被分给了负载较大的机器，就交换这两个待分配作业。反复调整后，剩余作业按长度从大到小，依次分给原负载从小到大的机器，工时仍不超过 T。LPT 也先把最大的 m 个作业各放一台，之后每次选择负载最小的机器。只要它没有给任何机器安排第三个作业，就与上面得到的方案一致。

最后证明 LPT 确实不会安排第三个作业。假设它第一次准备把作业 p 分给一台已有两个作业的机器。由于每个作业都长于 $T/3$，这台机器当前的负载大于 $2T/3$；它又是当前负载最小的机器，所以其余机器的负载也都大于 $2T/3$。设此时还有 q 台机器各只有一个作业，那么这 q 个作业每个都长于 $2T/3$。在任何工时不超过 T 的方案中，它们都必须单独占一台机器，否则再加一个长于 $T/3$ 的作业就会超过 T。除去这 q 个作业，已经分配的还有 $2(m-q)$ 个，加上当前的 p，共 $2(m-q)+1$ 个。它们只能使用剩余的 $m-q$ 台机器，而每台至多容纳两个，显然放不下。这与存在工时不超过 T 的方案矛盾。因此，LPT 完成的正是上面得到的可行分配，工时不超过 T。 □

定理 11.11. 固定 m 时，LPT 的近似比为 $4/3 - 1/(3m)$。

证明. 将作业按长度从大到小编号，记 LPT 得到的工时为 M，最优工时为 OPT。取最终负载为 M 的一台机器，设分给它的最后一个作业是 j。由式 (11.2)，再利用所有作业的平均负载不超过 OPT，有 $M \leqslant OPT + \left(1 - \frac{1}{m}\right) l_j$。因此，只要 $l_j \leqslant OPT/3$，就能得到所需的上界。下面说明，如果 $l_j > OPT/3$，LPT 实际上已经得到了最优方案，从而定理得证。

暂时只考虑前 j 个作业，它们的长度都大于 $OPT/3$。在原问题的最优方案中删去编号大于 j 的作业后，各台机器的负载只会减小，所以这前 j 个作业一定能在 OPT 时间内完成。由引理 11.10，LPT 安排这前 j 个作业时，工时也不超过 OPT。注意 j 后面的作业是否存在，不会影响 LPT 前 j 个作业的分配。原来选定的那台机器在分到作业 j 后，就不再接收其他作业了，此时它的负载已经是 M。既然处理完前 j 个作业时所有机器的负载都不超过 OPT，就有 $M \leqslant OPT$，即 LPT 已经最优。

最后给出达到这一比值的例子。$m = 1$ 时，LPT 自然最优。对于 $m > 1$，取长度为 $m+1, m+2, \ldots, 2m-1$ 的作业各两个，再取三个长度为 m 的作业。这些作业的总长度为 $3m^2$，因此任何方案的工时都至少为 $3m$。把三个长度为 m 的作业交给同一台机器，其余作业按一长一短配对，使每对长度之和为 $3m$，便能让每台机器的负载都等于 $3m$，所以 $OPT = 3m$。若使用 LPT，不难计算得到工时为 $4m - 1$。因此，LPT 的近似比确实可以达到 $\frac{4m-1}{3m} = \frac{4}{3} - \frac{1}{3m}$。 □

11.3 装箱问题

提到近似算法，不可能不提到装箱这一经典问题。假设有若干件物品，需要把它们全部放进容量相同的箱子，每件物品都必须完整地放在一个箱子里，怎样安排才能用最少的箱子？本节讨论一维装箱，即物品的尺寸是一维的。把箱子容量记为 1，就可以将问题写成：给定 n 件大小为 $a_i \in (0, 1]$ 的物品，将它们分组，使每组的大小之和不超过 1，并让组数尽可能少。

更一般的装箱问题还可能涉及物品的形状、到达顺序等条件，常见的变体如下。

到达方式	空间维数	形状约束	优化目标
全部物品预先给定；物品逐件到达	一维容量；二维矩形；三维长方体	规则形状；不规则形状；是否允许旋转	装完物品用最少箱子；固定箱子装入最多价值

11.3.1 装箱问题的困难性

定理 11.12. 给定大小为有理数 $a_i \in (0, 1]$ 的物品，判断它们能否装入两个单位容量的箱子，是 NP 完全问题。

证明. 我们再次从划分问题归约。给定正整数 $c_1, \ldots, c_n$，记它们的总和为 S，要判断能否把它们分成和相等的两组。若某个 $c_i > S/2$，这个划分实例一定无解，将它转换为三个大小均为 1 的物品即可，它们同样装不进两个箱子。其余情况下，令 $a_i = \frac{2c_i}{S}$。每件物品的大小都在 $(0, 1]$ 内，而且总大小为 2。因此，两个箱子能够装下它们，当且仅当两个箱子都恰好装满，这又等价于原来的整数可以分成和均为 $S/2$ 的两组，从而得到了多项式时间归约。另一方面，给定每件物品所在的箱子，只需分别求和就能验证容量限制，所以这个判定问题属于 NP，从而是 NP 完全的。 □

上面的证明和最小化工时调度问题类似，其实最小化工时调度问题可以视为装箱个数固定最小化最大箱子中的物品大小的问题，因此和固定箱子最大的容积求最小化箱子个数的问题“对偶”。

定理 11.13. 除非 P = NP，否则装箱问题不存在近似比小于 3/2 的多项式时间算法。

证明. 假设存在这样的算法，近似比为 $r < 3/2$。用它处理上面构造的总大小为 2 的实例。如果两个箱子能装下，则 $OPT = 2$，算法返回的箱数至多为 $2r < 3$。箱数是整数，又不可能少于最优值，因此算法必须恰好使用两个箱子。如果两个箱子装不下，任何可行方案都至少用三个箱子，算法也一样。故只看算法是否用了两个箱子，就能在多项式时间内解决划分问题，除非 P = NP，否则不可能。 □

这个证明的关键是，近似保证把答案限制在 2 与 3 之间，而整数答案只能是 2，于是近似算法被迫给出最优解。后面介绍的 FFD 算法恰好能达到 3/2 的绝对近似比。

11.3.2 Fit 算法

Fit 算法是一系列算法的总称，每种 Fit 算法按照各自给定的规则逐件装箱。下面先看最简单的 Next Fit（NF），再讨论利用已有箱子剩余空间的几种办法。

Next Fit

Next Fit 从一个空箱子开始，依次往里面装物品。遇到放不下的物品，就关闭当前箱子，另开一个箱子来装它。已经关闭的箱子以后不再使用，因此算法每次只需检查当前箱子的剩余空间。

定理 11.14. 对非空输入，$NF(I) \leqslant 2OPT(I) - 1$，其渐近近似比为 2。

证明. 记箱子 B_i 的最终装载量为 $s(B_i)$。打开 B_{i+1} 的第一件物品放不进 B_i，故 $s(B_i) + s(B_{i+1}) > 1$。若算法用了至少 $2OPT$ 个箱子，把前 $2OPT$ 箱两两配成 $(B_1, B_2), (B_3, B_4), \ldots$，总物品大小就大于 OPT，不可能装入 OPT 个单位箱，矛盾。

再构造一组比值趋于 2 的输入：取偶数 m，输入 m 组 $(1/2, 1/m)$。NF 每组开一箱，共 m 箱。最优方案把大小为 $1/2$ 的物品两两装箱，再把所有大小为 $1/m$ 的物品装满一箱，恰需 $m/2 + 1$ 箱。因此比值 $m/(m/2 + 1)$ 随 m 增大趋于 2，并且任何小于 2 的系数加固定常数也无法覆盖这族实例。 □

Any Fit

NF 一旦关闭某个箱子，即使后来有物品能放进去，也不再使用它，剩余空间就浪费了。Any Fit 要求先检查已有箱子，只有目前所有箱子都装不下新物品时，才新开箱。其中三个常见算法如下。如果有多个箱子满足条件，则选择最早开启的箱子。

- First Fit（FF）：选最早开启的箱子。
- Best Fit（BF）：选剩余空间最小的箱子。
- Worst Fit（WF）：选剩余空间最大的箱子。

Any Fit 只有在已有箱子都放不下时才开新箱，所以较晚箱子的第一件物品必定放不进较早箱子。这说明任意两箱的最终装载量之和都大于 1，沿用 NF 的证明，就能得到非空输入上 $2OPT - 1$ 的共同上界。WF 在前面大小为 $1/2, 1/m$ 的交替输入上仍然每组开一箱，所以它的渐近近似比也为 2。

从定义来看 BF 算法似乎比 FF 算法有更高的箱子利用率，但并非总是如此。下面的例子说明不会存在“BF 总比 FF 好”或者“FF 总比 BF 好”的结论：输入 $0.5, 0.7, 0.1, 0.4, 0.3$ 时，FF 得到 $(0.5, 0.1, 0.4)$、$(0.7, 0.3)$，共两箱，BF 则得到 $(0.5, 0.4)$、$(0.7, 0.1)$、(0.3)，共三箱。输入 $0.5, 0.7, 0.3, 0.5$ 时，FF 得到 $(0.5, 0.3)$、(0.7)、(0.5)，共三箱，BF 得到 $(0.5, 0.5)$、$(0.7, 0.3)$，共两箱。

Almost Any Fit

FF 与 BF 都有一个 WF 不具备的性质。按开启次序给箱子编号，在剩余空间最大的箱子中，记最晚打开的一个为 B_k。只要当前物品能放进 $B_1, \dots, B_{k-1}$ 中的某个箱子，它们就不会选择 B_k。满足这一要求的 Any Fit 算法称为 Almost Any Fit（AAF）。FF 总是先找较早的箱子，自然满足要求；BF 总是先找更满的箱子，剩余空间相同时又优先选较早的箱子，因此也满足要求。

WF 不满足这一限制。比如先来 $0.7, 0.4$，再来 0.1，WF 把最后一件放进第二箱，虽然第一箱也能装下。我们可以把 WF 改为将当前物品放入能装下它的剩余空间第二大的箱子中；若这样的箱子不存在，便将其放入能装它的剩余空间最大的箱子中；若上述两种情况都不存在，则新打开一个箱子将其装入。这一算法称为 Almost Worst Fit（AWF），不难理解其满足 AAF 的要求。

FF、BF 和 AWF 都属于 AAF。Johnson 证明了这类算法的渐近近似比至多为 $17/10$，比 NF、WF 的 2 更好。补充材料将用权重分析证明 FF 的具体上界 $FF(I) \leqslant \frac{17}{10}OPT(I) + \frac{4}{5}$。

First Fit Decreasing

如果允许先把全部物品按大小非增排序，再调用 FF，就得到 First Fit Decreasing（FFD）。

> **定理 11.15.** FFD 的近似比为 $3/2$。

证明. 设算法用了 b 箱。$b = 0, 1$ 时结论直接成立，以下设 $b \geqslant 2$。令 $j = \lceil 2b/3 \rceil$，考察第 j 箱 B_j。如果 B_j 中有大于 $1/2$ 的物品，那么开启它及之前各箱的第一件物品都不小于这件物品，故至少 j 件物品在任何方案中都必须分开，故 $OPT \geqslant j \geqslant 2b/3$。否则，开启第 j 箱的物品不超过 $1/2$。由于物品已经按大小递减排列，之后处理的物品也都不超过 $1/2$，所以第 j 箱及之后各箱中的物品均满足这一条件。其中除最后一箱外，每箱至少有两件。因此，第 j 箱到最后一箱中的物品数至少为 $2(b-j)+1 \geqslant j-1$，其中不等式来自于 $3\lceil 2b/3 \rceil \leqslant 2b+2$。从第 j 箱及之后的箱子中挑出 $j-1$ 件不同的物品 $x_1, \dots, x_{j-1}$，分别与前面的箱子 $B_1, \dots, B_{j-1}$ 配对。每件 x_i 到来时都放不进前 $j-1$ 个箱子，箱子的装载量以后只增不减，故 $s(B_i) + x_i > 1$。这些配对使用的物品互不重叠，于是全部物品的大小至少为 $\sum_{i=1}^{j-1}\big(s(B_i) + x_i\big) > j-1$。因此单位箱至少需要 j 个，即 $OPT \geqslant j \geqslant 2b/3$。

最后取物品 $1/2, 1/4, 1/4, 1/3, 1/3, 1/3$。FFD 排序后装成 $(1/2, 1/3)$、$(1/3, 1/3, 1/4)$、$(1/4)$，需要三箱；最优是 $(1/2, 1/4, 1/4)$ 和 $(1/3, 1/3, 1/3)$，只要两箱，达到 $3/2$。 □

FFD 还有更强的保证 $FFD(I) \leqslant (11/9)OPT(I) + 2/3$，这一界给出了更小的渐近近似比。Dósa（2007）给出达到这一式子的构造，Dósa、Li、Han、Tuza（2013）给出完整上界证明。下面展开达到等号的实例，完整上界证明比较复杂，读者可参考原文。

固定 $\eta = 1/1000$，记四类物品为 $A = \frac{1}{2} + \eta$，$B = \frac{1}{4} + 2\eta$，$C = \frac{1}{4} + \eta$，$D = \frac{1}{4} - 2\eta$。对任意整数 $t \geqslant 0$，取 A, B, C 三类物品各 $6t+4$ 件，D 类物品 $12t+8$ 件。先给出一个最优装法：$6t+4$ 箱装 (A, C, D)，$3t+2$ 箱装 (B, B, D, D)。每箱都恰好装满，$OPT = 9t + 6$。

FFD 按 A, B, C, D 的顺序处理。每个 A 后装一个 B，余下空间放不下 C 或 D；C 三件一箱，剩一件；最后的 D 先给这个单独的 C 配三件，再每四件一箱，最后剩一件。具体计数为

箱内物品	A, B	C, C, C	C, D, D, D	D, D, D, D	D
箱数	$6t+4$	$2t+1$	1	$3t+1$	1

共 $11t + 8 = (11/9)(9t + 6) + 2/3$ 箱。

图 11.1 汇总了 NF、FF、BF 和 FFD 处理同一组物品时的装箱结果。

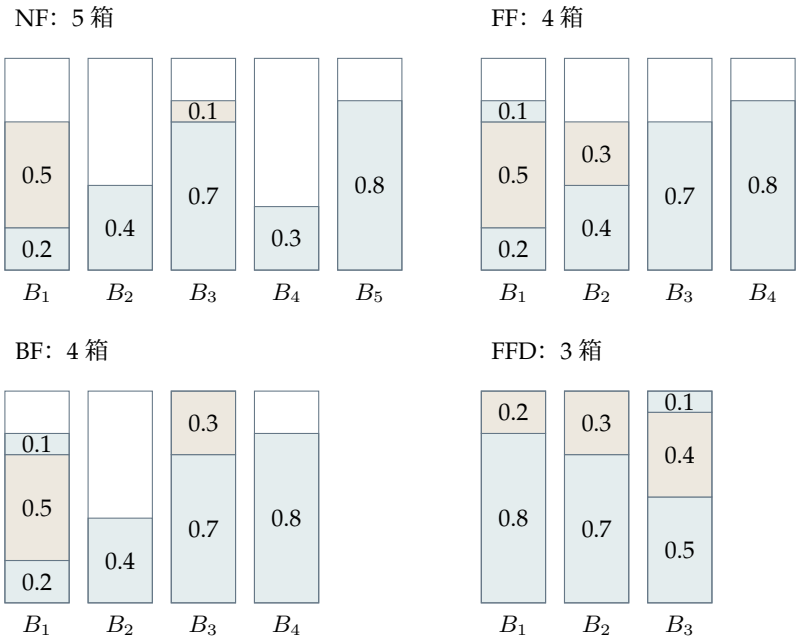


图 11.1 同一输入 $0.2, 0.5, 0.4, 0.7, 0.1, 0.3, 0.8$ 的四种装法。

11.3.3 在线装箱算法

前面的 FFD 要先知道所有物品才能排序，这种允许预先查看全部输入的算法称为离线算法。如果物品像传送带上的货物一样逐件到达，每件到来时必须立即决定放在哪一箱，而且以后不能搬动，就需要在线算法。

我们仍把算法使用的箱数 $A(I)$ 与知道全部输入时的最少箱数 $OPT(I)$ 比较。若对每个输入都满足 $A(I) \leqslant rOPT(I)$，就称算法具有 r 的绝对竞争保证，所有可取的 r 的下确界称为绝对竞争比。它衡量在线决策与预先知道全部输入的最优方案相差多少。即使给在线算法无限的计算时间，它也可能因为不知道后续物品而无法做出最优选择。例如，先来两件 0.4。若分开放，此时结束输入，算法用两箱而最优只需一箱；若放在一起，再来两件 0.6，算法用三箱，最优是两组 $(0.4, 0.6)$。任何确定性在线算法都会遇到其中一种情况。这个构造根据算法如何放置前两件物品，决定后续输入，直观而言是在针对算法的弱点“对抗”算法。

NF、FF、BF 和 WF 都可以在物品到达时立即决定放在哪一箱，因此都是在线算法。FFD 需要先排序，必须预先知道全部输入，所以是离线算法。

上面的例子已经说明，在线算法可能因为不知道后续物品而多用箱子。继续利用“根据算法决定接下来送来哪些物品”的办法，还能证明下面这个更强的下界。

定理 11.16. 任何确定性在线装箱算法的绝对竞争比至少为 $5/3$。

证明. 令 $\eta = 1/1000$，三种物品大小分别为 $s = \frac{1}{7} + \eta,\quad t = \frac{1}{3} + \eta,\quad u = \frac{1}{2} + \eta$。先输入六件 s，它们能放入一个箱子。若算法已用至少两箱，就在这里停止输入，得到比值至少为 2。否则它们必全在一箱，此时再输入六件 t。原箱剩余空间小于 t，每个新箱最多放两件 t。如果此时已用至少五箱，就停止输入：这十二件物品可以用三箱装完，其中每箱两件 s、两件 t；而全部物品大小大于 2，故 $OPT = 3$，比值至少 $5/3$。剩下的情况只能是算法用了四箱，即一箱六件 s，另三箱各两件 t。最后输入六件 u，每件都放不进旧箱，也不能与另一件 u 同箱，只能再开六箱，共十箱。可以每箱放一件 s, t, u，得到 6 个箱子，又因为六件 u 必须分开，$OPT = 6$。此时比值为 $10/6 = 5/3$。 □

Balogh 等（2015）给出了绝对竞争比为 $5/3$ 的在线算法，恰好达到这个下界。因此，确定性在线装箱的最优绝对竞争比为 $5/3$。

11.4 背包问题

11.4.1 基础的 2-近似算法

在贪心算法一讲中，我们已经讨论过分数背包的解法，即优先装入单位重量价值最高的物品。对于 0-1 背包，这样做还能得到最优解吗?

考虑容量为 3 的背包和两件物品，第一件的重量、价值都是 1，第二件的重量为 3、价值为 2。按单位重量价值排序，就会先拿第一件，随后第二件放不下，最后只得到价值 1。最优方案却是直接拿第二件，得到价值 2。如果把容量和第二件的重量都改成 C，第二件的价值改成 $C-1$，贪心仍只拿第一件，而最优值变成 $C-1$。因此，仅靠分数背包的贪心算法，连固定常数的近似保证也得不到。

改进的方法很简单。在分数背包贪心算法给出的方案之外，再考虑只拿价值最大的那件物品，最后选择两者之中价值较大的方案。

定理 11.17. 对于物品重量为正、价值非负的 0-1 背包问题，上述算法是 2-近似的。

证明. 记容量为 C，物品 i 的重量、价值分别为 $w_i > 0$ 和 $v_i \geqslant 0$。先删去单独也放不进背包的物品，再将其余物品按 v_i/w_i 从大到小排列。若全部物品都能放下，全部选入就是最优解；否则，设第 j 件是第一件放不下的物品，前面已经装入的物品集合为 $P = \{1, \ldots, j-1\}$，总价值记为 $v(P)$。

回到允许切开物品的分数背包。它会先装入 P，再取第 j 件的一部分。任何 0-1 背包方案都能用于分数背包，所以分数背包的最优值 P_{frac} 不小于 OPT。如果把最后取的那部分补成完整的一件，只会使价值增加，因此

$$OPT \leqslant P_{\text{frac}} \leqslant v(P) + v_j \leqslant v(P) + v_{\max},$$

其中 $v_{\max}$ 是所有能单独装下的物品中的最大价值。算法在 P 和最大价值单件中选较好者，返回价值 $A = \max\{v(P), v_{\max}\}$，于是 $OPT \leqslant v(P) + v_{\max} \leqslant 2A$。两个候选方案都能装进背包，这就证明了算法的 2-近似保证。 □

我们也可以在遇到装不下的物品后，跳过它，继续尝试后面的物品。这只会在原来已经选中的物品之外再增加一些物品，总价值不会减少，因此最后再与最大价值单件比较，仍有 2-近似保证。

此外，我们可以找到一组输入，使最优价值与算法返回价值的比值任意接近 2。取容量 4，三件物品的 (w, v) 为 $(2+\eta, 2+2\eta), (2,2), (2,2)$，其中 $0 < \eta < 1/2$。第一件密度最大且单件价值最大，算法得到 $2+2\eta$；最优拿后两件得到 4，比值趋于 2。

11.4.2 FPTAS 算法

0-1 背包问题的经典动态规划算法需要 $O(nC)$ 时间，其中 C 是背包容量，因此是伪多项式时间算法。这一小节我们希望给出一个 FPTAS，我们的想法是，如果容量 C 很大，但物品价值较小，可以改用总价值作为数组下标，记录达到这个价值至少需要多少重量。求出这些最小重量后，再选出重量不超过容量的最大价值即可。在接下来的讨论中，我们假设重量、价值和容量都是非负整数，且物品重量为正。此外，我们先删去重量超过容量的物品，并将剩余物品中的最大价值记为 $v_{\max}$。如果没有剩余物品，或者 $v_{\max} = 0$，直接返回空集即可。下面只讨论 $v_{\max} > 0$ 的情况。

记 $D[i, p]$ 为从前 i 件物品中选出总价值恰好为 p 的物品时，所需的最小重量。如果无法凑出这个价值，就记为 $+\infty$。不难得到递推式

$$D[i,p] = \begin{cases} D[i-1,p], & p < v_i, \\ \min\{D[i-1,p],\ w_i + D[i-1, p-v_i]\}, & p \geqslant v_i. \end{cases} \tag{11.3}$$

初值为 $D[0,0] = 0$，$D[0,p] = +\infty (p > 0)$。按 i 从小到大填表后，找出满足 $D[n,p] \leqslant C$ 的最大 p，就是背包能取得的最大价值。记 $Q = \sum_i v_i$，那么算法的时间复杂度为 $O(nQ)$。由于 $Q \leqslant nv_{\max}$，因此时间复杂度为 $O(n^2 v_{\max})$，因此只要 $v_{\max}$ 的大小是 n 的多项式级别的，就可以得到一个多项式时间的算法。然而并非所有实例都能满足这一条件，因此需要对输入的价值做一些技术性的处理，即对输入的所有价值做同比例的缩小，使得 $v_{\max}$ 能缩小到 n 的多项式级别。

如果把所有价值都除以同一个正数 b，任意两个物品集合的价值大小关系都不会改变，因此最优选择也不会改变。不过，v_i/b 未必是整数，无法直接作为数组下标，所以我们还需要取整。这里使用向上取整，令 $q_i = \lceil v_i/b \rceil$，再以 q_i 作为物品价值运行动态规划。重量和容量保持不变，得到的物品集合仍能装进原来的背包。当然，取整可能改变最优选择，但误差可以估计。把 q_i 乘回 b，记为 $\widetilde{v}_i = bq_i$，便有 $v_i \leqslant \widetilde{v}_i < v_i + b$。也就是说，每件物品的价值最多增加不到 b，一个方案至多选 n 件，总误差就不到 nb。下面根据想达到的精度选择 b，让这个误差足够小。

定理 11.18. 令 $0 < \varepsilon \leqslant 1$，$v_{\max} > 0$，取

$$b = \frac{\varepsilon v_{\max}}{n}.$$

用动态规划 11.3 对 $q_i = \lceil v_i/b \rceil$ 精确求解，得到集合 S。再把 S 与原价值最大的单个物品比较，返回价值较大者。所得算法是 $(1+\varepsilon)$-近似算法。

证明. 记原问题的最优物品集合为 S^*。改变价值时没有改变重量，所以 S^* 仍然是新问题的可行选择。动态规划选出的 S 使 $\sum q_i$ 最大，也就使 $\sum \widetilde{v}_i = b\sum q_i$ 最大，因此 S 的舍入后总价值不小于 S^* 的。再利用每件物品增加的价值不到 b，有

$$OPT = v(S^*) \leqslant \sum_{i \in S^*} \widetilde{v}_i \leqslant \sum_{i \in S} \widetilde{v}_i \leqslant v(S) + nb = v(S) + \varepsilon v_{\max}.$$

返回集合的真实价值 $A = \max\{v(S), v_{\max}\}$，于是 $OPT \leqslant A + \varepsilon A = (1+\varepsilon)A$。由于 S 与最大价值单个物品都能装入背包，返回方案同时满足容量要求。 □

与最大价值单个物品比较，是为了保证返回价值至少为 $v_{\max}$。来看一个缩放解未满足这个条件的例子。取容量 2，重量 $(2,1,1)$，价值 $(100,49,49)$，$\varepsilon=1$。此时 $b=100/3$，缩放价值为 $(3,2,2)$。后两件的缩放价值之和为 4，大于第一件的 3，所以动态规划会选后两件；但它们原价值之和只有 $98<100=v_{\max}$。与价值为 100 的第一件物品比较后，算法会改为返回它，保证证明中所用的 $v_{\max}\geqslant A$。

因为 $v_i\leqslant v_{\max}$，有 $q_i\leqslant\lceil\frac{n}{\varepsilon}\rceil$，$Q\leqslant n\lceil\frac{n}{\varepsilon}\rceil\leqslant\frac{n^2}{\varepsilon}+n$。所以时间复杂度为 $O(n^3/\varepsilon)$。这一上界关于 n 和 $1/\varepsilon$ 都是多项式，从而我们得到了 0-1 背包问题的 FPTAS。

注意，这里的近似方案依赖两个事实。首先，改变价值不会使所选物品超出容量，因此求出的方案仍可用。其次，每件物品的误差不到 b，总共至多选 n 件，所以总误差不到 nb。把舍入方法用于其他问题时，也需要分别证明方案仍然可行，以及误差能控制到什么程度。

11.4.3 关于 FPTAS 的一些讨论

背包问题有 FPTAS，但有些 NP 困难问题无法做到这一点。原因在于，如果可行解的目标值都是整数，只要把误差控制在 1 以内，近似解就必须是最优解。例如，装箱问题如果最优方案需要 10 个箱子，而算法保证使用的箱数少于 11，那么它就只能使用 10 个箱子。具体来说，设输入的二进制编码长度为 n，最优值满足 $0\leqslant OPT\leqslant B(n)$，其中 $B(n)\geqslant 1$ 是一个多项式。如果存在 FPTAS，就取 $\varepsilon=1/(2B(n))$，并将返回方案的目标值记为 A。对于最小化问题，有 $0\leqslant A-OPT\leqslant\varepsilon OPT\leqslant\frac{1}{2}$；对于最大化问题，有 $0\leqslant OPT-A\leqslant\varepsilon A\leqslant\frac{1}{2}$。由于 A 和 OPT 都是整数，两种情况下都只能有 $A=OPT$。而 FPTAS 的时间关于 n 和 $1/\varepsilon$ 都是多项式，这里 $1/\varepsilon=2B(n)$ 仍是 n 的多项式，所以这样就得到了多项式时间的精确算法。因此，除非 P = NP，满足上述条件的 NP 困难问题没有 FPTAS。

装箱就是一个直接的例子。若有 n 件物品，每件都能单独装入一箱，则 $OPT\leqslant n$。若有 FPTAS，取 $\varepsilon=1/(2n)$，它比最优方案多用的箱数至多为 $1/2$，所以必须恰好用 OPT 箱。这就能在多项式时间内精确解决装箱问题，与 NP 困难性矛盾，除非 P = NP。

11.5 聚类与中心选择

假设有 n 个居民点，要建 k 个服务中心。每个居民点去最近的中心，我们希望最远的居民也不用走太远。形式化而言，给定点集 S 和距离 d，对中心集合 C 定义 $d(s,C)=\min_{c\in C}d(s,c)$，$r(C)=\max_{s\in S}d(s,C)$。目标是用至多 $k(1\leqslant k\leqslant n)$ 个中心，使覆盖半径 $r(C)$ 最小。

用 X 表示居民点所在空间，则居民点集 $S\subseteq X$，中心集合 $C\subseteq X$。我们要求距离满足度量性质，即 $d(x,y)\geqslant 0$，$d(x,y)=0\iff x=y$，$d(x,y)=d(y,x)$，$d(x,y)\leqslant d(x,z)+d(z,y)$。记最优覆盖半径为 r^*。

11.5.1 逐个添加最佳中心

一种自然的想法是：先把第一个中心放在解决单中心问题的最佳位置，再每次增加一个中心，使当前半径尽量减小。

例如，直线上有四个居民点 $S=\{-10,-9,9,10\}$，要设置两个中心。如果只设一个中心，最佳选择是 0，覆盖半径为 10。但保留这个中心后，无论第二个中心选在哪里，都不能把半径降到 10 以下。因为两端的 -10 和 10 到原中心的距离都是 10，若要同时缩短它们到中心的距离，就必须由新中心同时覆盖它们，而这两个点相距 20，一个中心无法以小于 10 的半径覆盖两者。

如果直接把两个中心设在 -9.5 和 9.5，分别服务左右两组居民点，半径只需 0.5。这也是最优方

案，因为任意两个居民点至少相距 1，半径小于 0.5 的一个中心至多覆盖一个居民点，两个中心无法覆盖全部四个点。两种方案的半径相差 20 倍。进一步，把居民点改成 $-D, -D+1, D-1, D$ $(D \geqslant 2)$，上述方法的半径为 D，最优半径仍为 0.5，比值为 $2D$。因此，逐个添加最佳中心甚至无法保证一个固定的近似比。

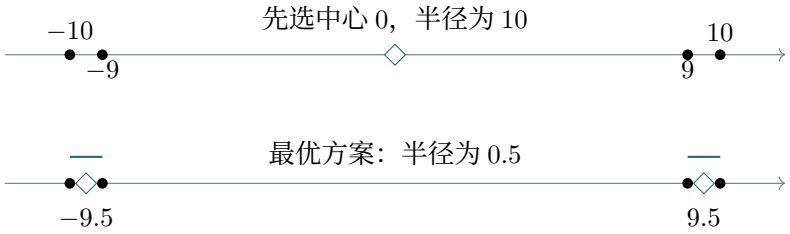


图 11.2　逐个添加最佳中心无法得到最优解。

11.5.2　思维实验：如果我们已知最优半径？

当然，直接寻找最优中心是一件很困难的事情。一个可行的解法来源于一个思维实验：假设已经知道最优解 r^*（也许是神谕），这是否会有帮助呢？这一利用思维实验，假设已知最优解并基于此构造算法的方式在近似算法设计中也是常见的。然而在知道了最优解 r^* 后，如果任意放 k 个中心，以 r^* 为半径，很有可能仍然做不到将所有点覆盖。我们将所有中心都放在居民点上，然后考虑这样一个事实：在知道了最优解 r^* 后，某个点 s 到最优解中最近的中心的距离是不超过 r^* 的，既然任意选取 s 为中心时用 r^* 为半径无法覆盖，那么如果放大半径，将真正最优解为中心，r^* 为半径的圆内所有点都囊括进来，那不就还是可以实现覆盖吗？

接下来的问题是，半径需要放到多大。事实上结论是显然的：在半径设置为 $2r^*$ 时就可以实现。为了理解这一点，任取一个居民点 s。最优方案中必有中心 c^* 距它不超过 r^*。我们不知道 c^* 在哪里，但可以直接把 s 当中心。原来由 c^* 服务的任一点 x 都满足 $d(s,x) \leqslant d(s,c^*) + d(c^*,x) \leqslant 2r^*$。因此，虽然不知道 c^* 在哪里，只要把 s 设为中心，并允许半径达到 $2r^*$，就能覆盖原来由 c^* 服务的全部点。

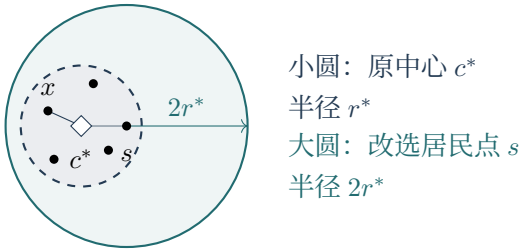


图 11.3　将中心改为居民点 s，半径扩大到 $2r^*$。

由此我们得到 Greedy-2r 算法：选一个还没覆盖的点作中心，删掉距它不超过 $2r^*$ 的点，再选下一个尚未覆盖的点。只要这一算法能在 k 步之内停止，就能得到一个覆盖半径不超过 $2r^*$ 的解。因此我们还需要说明算法能在 k 步之内停止。根据前面的讨论，如果最优解是 r^*，那么在算法中，每次随机选择一个剩余的点作为中心，$2r^*$ 为半径的圆至少会带走一个真正的最优解中的点为中心，r^* 为半径的圆内所有点，因此 k 步之后必然最优解覆盖的所有点都被算法覆盖，因此必然停止。

当然，我们并不能在解决问题之前就知道最优半径的长度，因此还需要通过二分查找来选择半径。根据上一段的讨论，如果用半径 r 试探时，Greedy-2r 算法在 k 步之内不停止，那么最优半径一

定大于 r，应当向较大的半径继续查找，否则再尝试较小的半径。二分的范围取 0 和所有居民点之间的最远距离即可。

11.5.3 最远点优先算法

当然我们也可以抛弃神谕来设计算法。一个很自然的想法是，我们可以首先任取一个居民点作为第一个中心，此后查看每个居民点到最近中心的距离，把距离最大的居民点选作下一个中心，直到选满 k 个。这样每一步都优先照顾离现有中心最远的居民点，这称为最远点优先算法。

> **定理 11.19.** 最远点优先算法的覆盖半径至多为 $2r^*$。

证明. 若返回半径 $R = 0$，结论成立。否则取最终距中心集合最远的点 x，距离为 R。考虑某一步加入的新中心 c，此前中心集合为 C'。由于算法选最远点，$d(c, C') \geqslant d(x, C') \geqslant d(x, C) = R$。因此，每个中心到之前选出的中心都至少相距 R，而 x 到最终所有中心也都至少相距 R。这就得到了 $k+1$ 个两两距离至少为 R 的点。最优方案只有 k 个中心，所以其中两个点必由同一个中心服务，它们到该中心的距离都不超过 r^*。根据三角不等式，这两个点之间的距离至多为 $2r^*$，所以 $R \leqslant 2r^*$。 □

11.5.4 近似比下界

前面给出的算法都能保证 2-近似，那么能不能把这个比值进一步降到 $3/2$，甚至更小呢？对于一般度量空间，下面的定理给出了回答。

> **定理 11.20.** 除非 P = NP，否则对于任何固定的 $1 \leqslant \rho < 2$，一般度量空间中的 k-center 问题都不存在多项式时间的 ρ-近似算法。

为了证明这一结论，需要引入一个 NP 完全问题，即支配集问题（dominating set）。给定图 $G = (V, E)$ 和整数 k，我们要判断能否选出至多 k 个顶点，使每个未选顶点都与某个选中顶点相邻。这样的顶点集合称为支配集。

证明. 给定一个支配集问题的实例 $G = (V, E)$ 和 k。只需考虑 $1 \leqslant k < |V|$，其余情况可以直接判断。把图的顶点作为居民点，整个空间也只包含这些顶点，并规定相邻顶点的距离为 1，不相邻顶点的距离为 2，即

$$d(u, v) = \begin{cases} 0, & u = v, \\ 1, & uv \in E, \\ 2, & u \neq v,\ uv \notin E. \end{cases}$$

这个距离满足度量性质。

以某个顶点为中心，半径 1 恰好能覆盖它自己及所有相邻顶点。因此，用至多 k 个中心、半径 1 覆盖全部居民点，恰好等价于选出至多 k 个顶点构成支配集。如果没有这样的支配集，半径 1 就不够，而半径 2 一定能覆盖全部点。所以，支配集存在时最优半径为 1，不存在时最优半径为 2。

现在假设有一个多项式时间的 ρ-近似算法，其中 $\rho < 2$。用它求解刚才构造的实例，会有以下两种情况。

1. 如果支配集存在，最优半径为 1，算法返回的半径至多为 $\rho < 2$。这个实例的半径只能是 1 或 2，所以算法必须返回 1。

2. 如果支配集不存在，最优半径为 2，任何可行方案的半径都只能是 2，算法也必须返回 2。

因此，只要看算法返回的是 1 还是 2，就能判断支配集是否存在。构造距离只需 $O(|V|^2)$ 时间，再运行一次近似算法，就得到了支配集问题的多项式时间算法。 □

11.6 旅行商问题

11.6.1 一般 TSP 的不可近似性

假如一位旅行商要到若干座城市推销商品，他需要从某座城市出发，访问其余每座城市一次，最后回到出发地。已知任意两座城市之间的路程，应该按什么顺序访问，才能让总路程最短？这就是旅行商问题（traveling salesman problem，TSP）。如果用顶点表示城市，用非负边权表示路程，就可以把它写成一个图上的问题：在给定的完全无向图中，寻找总权重最小的哈密顿回路。

上一讲已经说明 TSP 的判定版本是 NP 完全问题，优化版本是 NP 困难问题。自然地，我们希望像前面一样求近似解，但不限制边权时，连固定常数近似都做不到。

> **定理 11.21.** 除非 P = NP，否则对于任意固定常数 $k \geqslant 1$，TSP 都不存在多项式时间的 k-近似算法。

证明思想类似于定理 11.20，这里将哈密顿回路问题归约到 TSP 的 k-近似算法。

证明. 反证法。假设存在一个多项式时间的 k-近似算法 A，下面说明如何利用它在多项式时间内解决哈密顿回路问题。若 k 不是整数，把它向上取整后，A 仍满足相应的近似保证，因此以下设 k 为整数。

给定一个哈密顿回路问题的输入图 $G = (V, E)$，将其转化为一个 TSP 问题的实例，即在相同顶点上构造一个完全图 G'，将其边权重定义如下：

$$w(e) = \begin{cases} 1, & e \in E, \\ 2 + (k-1)n, & e \notin E, \end{cases}$$

其中 $n = |V|$。如果 G 中存在哈密顿回路，那么这条回路在 G' 中的每条边权重都是 1，总权重为 n，这也是对应 TSP 实例的最优值。如果 G 中不存在哈密顿回路，那么 G' 中的任何哈密顿回路都必须使用至少一条新增的边，所以总权重至少为 $(n-1) + \big(2 + (k-1)n\big) = kn + 1$。现在用算法 A 求解这个 TSP 实例，有以下两点：

1. 如果算法返回的回路总权重为 n，那么它的每条边权重都必须为 1，即全部来自原图 G，因此 G 存在哈密顿回路。
2. 如果 G 存在哈密顿回路，那么算法也必定返回总权重为 n 的回路。因为 A 是一个 k-近似算法，返回值至多为 kn；而任何使用新增边的回路，其总权重都至少为 $kn + 1$，不符合这一保证。

所以，检查算法是否返回总权重为 n 的回路，就能判断 G 是否存在哈密顿回路，且归约需要多项式时间。 □

11.6.2　添加三角不等式

尽管这一定理的结果十分悲观，但实际上这里给出的归约构造在现实问题中并不合理：现实中的 TSP 边的权重（也就是旅行商在两点之间通行的距离）应当满足下述三角不等式：

$$w(u,v) \leqslant w(u,x) + w(x,v),$$

其中 u, v, x 是图中的任意三个点。将权重满足三角不等式的 TSP 问题称为度量 TSP 问题。这让我们想起了上面的聚类问题，在那里也是通过度量空间的三角不等式限制从而允许了 2-近似算法的存在。那么在旅行商问题中，在添加度量空间的条件后，能得到怎样的结果呢?

> **定理 11.22** (倍树算法). 首先求最小生成树 T，然后从任意根开始，按深度优先遍历中顶点第一次出现的次序连接所有点，再回到起点。得到的路线代价不超过 $2w(T) \leqslant 2OPT$，其中 $w(T)$ 是最小生成树的总权重，OPT 是最优哈密顿回路的代价。

证明. 从最优旅行回路中删掉任意一条边，即可得到一棵生成树。由于边权非负，删边不会增加代价。因此 $w(T) \leqslant OPT$。

从根出发对树做深度优先遍历，每次沿树边走到孩子，遍历完它的子树后，再沿原边返回父亲。这样最终会回到根，每条树边都恰好走两次，得到总代价为 $2w(T)$ 的闭合路线 W。这条路线访问了所有顶点，但其中有些顶点出现了多次，接下来需要去掉这些重复访问。

把每个顶点除第一次以外的访问删掉，最后再保留回到根的一次访问。相邻保留顶点之间原来可能隔着一段走访，用一条直达边替代它。反复使用三角不等式，这条边的代价不超过被替代各边之和。把这些直达边的代价加起来，就不超过原来整条路线 W 的代价，因此新回路 H 满足

$$w(H) \leqslant w(W) = 2w(T) \leqslant 2OPT.$$

每个顶点恰好保留一次，图又是完全图，新路线确实是哈密顿回路。 □



图 11.4　相同的八个点，距离取欧氏距离。左为最小生成树，右为按首访次序连接后的回路。网格长度为 1。

图 11.4 的树边为 $ab, bc, bh, ad, de, ef, eg$。从 a 开始，完整走访是 $a, b, c, b, h, b, a, d, e, f, e, g, e, d, a$。保留首次访问再闭合，得到 $a, b, c, h, d, e, f, g, a$，树权重约为 11.893，完整走访约为 23.786，连接后回路约为 19.074。最优回路为 $a, b, c, h, f, g, e, d, a$，代价约为 14.715。

从上面的描述我们也能理解这一算法为什么称为倍树算法：这一算法把每条边都补了一份。这是让每个顶点度数变偶数的一种简单做法。若只给奇度顶点补上必要的边，可以进一步得到 3/2 的保证，补充材料中将介绍这一构造。

11.7 集合覆盖

为了引入集合覆盖问题，我们先从顶点覆盖问题说起。给定一张无向图，我们要选出尽可能少的顶点，使每条边至少有一个端点被选中。习题 11.6 将讨论这个问题的一个 2-近似算法。

现在换个角度来看这个问题。对于每个顶点，把所有以它为端点的边列成一个集合。选择这个顶点，就相当于选择了这个集合，其中的边也就全部被覆盖了。因此，要让每条边都被覆盖，只需选出若干个这样的集合，使它们的并集等于整个边集。要让选中的顶点尽可能少，就要让选中的集合尽可能少。

集合覆盖问题把这一想法推广到任意给定的集合，并允许选择不同集合的代价不同。设全部需要覆盖的元素组成集合 U，可选的非空子集为 $S_1, \ldots, S_m \subseteq U$，选择 S_i 的代价为 $c_i \geqslant 0$。我们希望选出其中一些集合，使它们的并集为 U，并使总代价最小。对于刚才的顶点覆盖，U 就是图的边集，每个可选集合对应一个顶点，代价均为 1。

选择集合时，既要考虑代价，也要考虑能新覆盖多少个元素。例如，一个集合的代价为 3，能新覆盖 6 个元素，另一个的代价为 1，只能新覆盖 1 个元素。平均到每个新覆盖的元素上，前者的代价为 0.5，小于后者的 1。记尚未覆盖的元素集合为 R，一个自然的想法是每次选择

$$\frac{c_i}{|S_i \cap R|}$$

最小的集合 S_i，其中只考虑 $|S_i \cap R| > 0$ 的集合。选中后，把这次覆盖的元素从 R 中删去，再按剩下的元素重新比较，直到 R 为空。如果最后还有元素没有任何集合能覆盖它们，就说明原问题无解。

上述贪心算法并不总能得到最优方案：设 $U = \{a, b, c, d, e, f, g, h\}$，可选集合有六个。其中 $S_1 = \{a, c, e, g\}$，$S_2 = \{b, d, f, h\}$，各自的代价为 1.1。另外四个集合 $\{a\}$、$\{b\}$、$\{c, d\}$、$\{e, f, g, h\}$ 的代价均为 1。若使用贪心算法，第一步选择 $\{e, f, g, h\}$，第二步选择 $\{c, d\}$，最后分别选择 $\{a\}$ 和 $\{b\}$，总代价为 4。而直接选择 S_1 和 S_2 就能覆盖全部元素，总代价为 2.2，更优。因此，每一步按平均代价选择未必能让最后的总代价最少。不过，下面可以证明它与最优值之间仍有一个上界。

定理 11.23. 设 U 非空且存在可行覆盖，$d = \max_i |S_i|$ 为一个集合最多包含的元素数。上述贪心算法的总代价至多为 $H_d OPT$，其中

$$H_d = 1 + \frac{1}{2} + \cdots + \frac{1}{d} \leqslant 1 + \ln d.$$

证明. 每次选中一个集合后，把选择它的代价平均分摊给本次新覆盖的元素。若这一轮选中 S_i，轮开始时未覆盖的元素集合为 R，则每个新覆盖的元素分摊到的代价为 $c_i/|S_i \cap R|$。将元素 x 分摊到的代价记为 a_x。每个元素只在首次被覆盖时参与分摊，因此算法的总代价 A 满足 $A = \sum_{x \in U} a_x$。我们先证明，对任意一个可选集合 S_i，分给它内部所有元素的代价之和，至多为直接选择这个集合的代价 c_i 的 H_d 倍。之后把这个结论用到最优方案选择的集合上，就能比较 A 与 OPT。

设 S_i 有 t 个元素，按它们被算法首次覆盖的先后顺序记为 $x_1, \ldots, x_t$，其中同一轮覆盖的元素任意排序。考虑 x_j 被覆盖的那一轮。在这一轮开始时，$x_j, \ldots, x_t$ 都还未覆盖，因此若选择 S_i，至少能新覆盖 $t - j + 1$ 个元素，平均代价至多为 $c_i/(t - j + 1)$。算法实际选择的集合可能是另一个，但它的平均代价不会比选择 S_i 更高，所以 $a_{x_j} \leqslant \frac{c_i}{t-j+1}$。对 S_i 中的全部元素求和，得到

$$\sum_{x \in S_i} a_x \leqslant c_i \left(\frac{1}{t} + \cdots + \frac{1}{2} + 1 \right) = c_i H_t \leqslant c_i H_d.$$

现在取一个最优方案，用 J^* 表示它选中的集合下标。这些集合覆盖了 U，因此对这些集合分别求出其中所有元素分摊的代价，再相加，就至少包含了每个元素的 a_x 一次。由于 $a_x \geqslant 0$，重复计算只会使总和增大。于是 $A = \sum_{x\in U} a_x \leqslant \sum_{i\in J^*} \sum_{x\in S_i} a_x \leqslant H_d \sum_{i\in J^*} c_i = H_d OPT$。 □

11.8 补充材料

11.8.1 FF 的渐近近似比

设 FF(I) 表示对 I 运用 FF 得到的装箱数。我们证明，对于任意实例 I，有

$$FF(I) \leqslant 1.7OPT(I) + \frac{4}{5},$$

这可以说明 First Fit 的渐近近似比至多为 1.7。这里的证明需要使用摊还分析里面的“均摊”的思想。设第 i 件物品体积为 a_i，定义权重 $w(a_i)$，其可视为摊还的体积。记 $w(I)$ 为装箱的一个实例 I 的权重总和，则对实例 I，应有

$$w(I) = \sum_{i=1}^{n} w(a_i) = \sum_{j=1}^{FF(I)} w(B_j) = \sum_{j=1}^{OPT(I)} w(B_j^*),$$

其中 B_j 是 FF 打开的第 j 个箱子，B_j^* 是最优方案中的第 j 个箱子，$w(B)$ 表示箱子 B 内全部物品的权重之和。此时如果能满足：（1）$w(I) \leqslant 1.7OPT(I)$；（2）$FF(I) \leqslant w(I) + \frac{4}{5}$，就证明了结论。基于上面的等式，对于第一条，我们其实就是希望对任意的 B_j^* 有 $w(B_j^*) \leqslant 1.7$；对于第二条，我们则希望有对任意的 B_j，在平均意义上差不多有 $w(B_j) \geqslant 1$。

接下来，我们定义一个特殊的权重函数（来自 Jiří Sgall 的工作）：

$$w(a_i) = \frac{6}{5}a_i + v(a_i),$$

其中，$\frac{6}{5}a_i$ 是放大后的体积（scaled size），$v(a_i)$ 是按物品大小额外计入的权重（bonus），定义为

$$v(a_i) = \begin{cases} 0, & a_i \leqslant \frac{1}{6} \\ \frac{3}{5}\left(a_i - \frac{1}{6}\right), & \frac{1}{6} < a_i \leqslant \frac{1}{3} \\ \frac{1}{10}, & \frac{1}{3} < a_i \leqslant \frac{1}{2} \\ \frac{2}{5}, & a_i > \frac{1}{2} \end{cases}$$

这个非负值函数 v 在 a_i 不超过 $\frac{1}{2}$ 时是连续的，它将物品按照尺寸分成了四类，我们称这四类物品依次为“极小的（tiny）”，“小的（small）”，“中等大小的（medium）”，“大的（big）”。

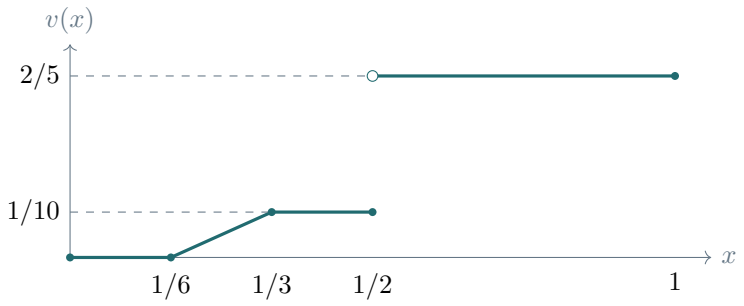


图 11.5 物品的附加权重 $v(a_i)$。实心点表示端点取值，空心点表示不取该值。

接下来便是渐近比的证明，根据前面的分析，将证明拆分成几个引理。

引理 11.24. 对于最优解的装箱 B_j^*，一定有 $w(B_j^*) \leqslant 1.7$。

证明. 每箱物品总体积至多为 1，所以 scaled size 之和至多为 $6/5$。接下来只需估计 bonus 之和。

- 如果箱子中不包含大的物品，由定义可知，对每件物品都有 $v(a_i) \leqslant 3a_i/10$。因此 bonus 之和至多为 $3/10$，箱子总权重至多为 $6/5 + 3/10 = 3/2 < 1.7$。
- 如果箱子中包含大的物品，这件物品的 bonus 为 $2/5$，剩余物品的总体积小于 $1/2$。其中至多两件物品的体积大于 $1/6$，也就是至多两件的 bonus 非零。若只有一件，它的 bonus 至多为 $1/10$；若有两件，记其体积为 x, y，则它们都小于 $1/3$，否则两件之和就大于 $1/2$。于是 $v(x) + v(y) = \frac{3}{5}\left(x + y - \frac{1}{3}\right) < \frac{1}{10}$。因此箱子总权重至多为 $6/5 + 2/5 + 1/10 = 1.7$。

□

可以看出，这个结论不依赖于最优解，它对所有可行解都成立。

引理 11.25. 记 $c(B_j)$ 代表 FF 解中箱子 B_j 内所有物品的尺寸和，$v(B_j)$ 代表 B_j 内所有物品的 bonus 和。如果两个权值不超过 1 的箱子 B_i 和 B_j 满足 B_i 在 B_j 之前打开 $(i < j)$，$\frac{2}{3} \leqslant c(B_i) < \frac{5}{6}$，且 B_j 中至少有两个物品，则：$\frac{6}{5}c(B_i) + v(B_j) \geqslant 1$。

证明. 设 B_j 中两个物品的尺寸为 x, y，并记 $c(B_i) = 5/6 - z\,(0 < z \leqslant 1/6)$。根据 FF 的规则，$B_j$ 中的物品到来时都放不进 B_i，所以 $x, y > 1 - c(B_i) = 1/6 + z$。由于 $1/6 + z \leqslant 1/3$，且 bonus 函数单调不减，有 $v(x), v(y) \geqslant v\,(1/6 + z) = 3/5z$。因此，后一箱中这两件物品的 bonus 足以补上前一箱 scaled size 距离 1 的差额，即

$$\frac{6}{5}c(B_i) + v(B_j) \geqslant \frac{6}{5}\left(\frac{5}{6} - z\right) + 2 \cdot \frac{3}{5}z = 1. \tag{11.4}$$

□

引理 11.26. 若 FF 至少使用两个箱子，则所有箱子的总权重加上 $4/5$ 后，不少于箱数，即

$$FF(I) \leqslant w(I) + \frac{4}{5}.$$

证明. 首先来看在什么情况下 $w(B_j) \geqslant 1$：

1. 如果 B_j 中包含的物品总体积不少于 $\frac{5}{6}$，则 $w(B_j) \geqslant 1$；
2. 如果 B_j 中包含了一个大的物品，则 $w(B_j) > \frac{6}{5} \times \frac{1}{2} + \frac{2}{5} = 1$；
3. 如果 B_j 中包含了两个中等大小的物品，那么 $w(B_j) \geqslant 2 \times \left(\frac{6}{5} \times \frac{1}{3} + \frac{1}{10}\right) = 1$。

换言之，对于那些权重之和小于 1 的箱子，一定都满足：(1) 没有大的物品；(2) 至多有一个中等大小物品；(3) 其中所有物品尺寸和小于 $\frac{5}{6}$。对于权重已经达到 1 的箱子，每箱都能用自身权重计入箱数。下面只考虑权重小于 1 的箱子，设共有 k 个。

1. 若 $k = 0$，所有箱子的权重都至少为 1，结论已经成立。

2. 若 $k=1$，把唯一权重不足 1 的箱子与任意另一个箱子放在一起。FF 的任意两个箱子总体积都大于 1，因为较晚箱子的第一件物品放不进较早的箱子。因此，这两箱的总权重大于 $6/5$，再加 $4/5$ 就大于 2。其余每箱的权重都至少为 1，所以总权重加 $4/5$ 足以计入全部箱数。
3. 若 $k \geqslant 2$，把这些权重小于 1 的箱子按开启顺序重新编号为 $B_1, \dots, B_k$。先看它们可能有哪些特殊情况。

   - 如果某箱只有一件物品，这件物品必定不大于 $1/2$，否则该箱权重已经大于 1。它之后的箱子中，每件物品都必须大于 $1/2$ 才放不进它，因而权重都大于 1。所以这样的单件箱至多有一个，而且只能是 B_k。
   - 如果某箱至少有两件物品，但总体积小于 $2/3$，那么它之后的每件物品都大于 $1/3$。后面若还有一个至少装两件物品的箱子，该箱就会含有两个中等物品或一个大物品，权重至少为 1。因此，这种体积小于 $2/3$ 的多件箱，只能是 B_{k-1} 或 B_k。

   由此可知，前 $k-2$ 箱的总体积都在 $[2/3, 5/6)$ 内，而 $B_2, \dots, B_{k-1}$ 都至少有两件物品。于是可以对 B_i, B_{i+1} 使用引理 11.25，把后一箱的 bonus 用来补足前一箱的 scaled size。最后两箱的总体积大于 1，它们合计的 scaled size 加上 $4/5$ 就超过 2。将这些不等式相加，得到

$$\begin{aligned}\sum_{i=1}^{k} w(B_i) + \frac{4}{5} &= \sum_{i=1}^{k-2} \left(\frac{6}{5} c(B_i) + v(B_{i+1}) \right) + \frac{6}{5}\big(c(B_{k-1}) + c(B_k)\big) + \frac{4}{5} + v(B_1) + v(B_k) \\ &\geqslant (k-2) + 2 = k.\end{aligned}$$

   当 $k=2$ 时，求和项中没有任何项，直接使用这两箱的估计即可。再把之前权重不小于 1 的箱子加回来，就得到结论。

□

将三个引理结合起来，就能得到开头要证明的结论。

> **定理 11.27.** 对任意装箱实例 I，$FF(I) \leqslant 1.7 OPT(I) + 4/5$，因此 FF 的渐近近似比至多为 1.7。

证明. FF 至少使用两箱时，引理 11.24 给出 $w(I) = \sum_{j=1}^{OPT(I)} w(B_j^*) \leqslant 1.7 OPT(I)$，引理 11.26 给出 $FF(I) \leqslant w(I) + 4/5$，合起来就得到所需上界。若没有物品，$FF = OPT = 0$；若 FF 只用一箱，则 $FF = OPT = 1$，不等式也成立。 □

下表汇总了装箱各经典算法的近似比。

算法	所用箱数的上界	渐近近似比
NF、WF	非空时至多 $2OPT - 1$	2
FF、BF	至多 $\lfloor 17OPT/10 \rfloor$	$17/10$
FFD	至多 $3OPT/2$	$11/9$

FF 和 BF 的绝对近似保证分别由 Dósa–Sgall（2013、2014）证明。前面的权重证明给出 $17OPT/10 + 4/5$；要进一步消去加性常数，还需对装箱情况做更细的分类，完整论证见这两篇论文。Johnson 对 AAF 的分析也给出了 $17/10$ 的渐近上界，证明使用了 AAF 的选箱条件。

11.8.2 Christofides 算法

旅行商问题的倍树算法已经给出了 2-近似，但它把生成树的每条边都走了两遍。这样做是为了保证能够走遍所有顶点并回到起点，如果能少重复一些边，就有机会缩短路线。Christofides 算法正是沿着这个思路，把近似比改进到了 $3/2$。

我们希望给最小生成树 T 加上一些边，使得到的图存在欧拉回路，也就是一条把每条边恰好走一次并回到起点的路线。连通无向图存在欧拉回路的条件是所有顶点的度数都为偶数。树已经连通，因此只需给度数为奇数的顶点各增加一条边。

记这些奇度顶点组成的集合为 O。图中所有顶点的度数之和等于边数的两倍，是偶数，因此奇度顶点一定有偶数个。把它们两两配对，再为每一对加一条边，就能使全部顶点的度数变为偶数。这种让每个顶点恰好与另一个顶点配对的边集称为完美匹配。为了使增加的代价最小，我们在 O 上求一个最小权完美匹配，记为 M（可以用带花树算法在多项式时间内求解，此处不展开具体算法）。

把 M 的边加入 T 后，每个奇度顶点的度数恰好增加 1，其余顶点的度数不变。如果匹配中的某条边已经在树里，就再保留一份这条边，两份分别计算度数和代价。这样得到的图仍然连通，而且所有顶点的度数都为偶数，所以可以找出一条欧拉回路。最后沿用倍树算法的处理，跳过已经访问过的顶点，用直达边连接前后保留的顶点，就得到了旅行商问题所需的回路。

定理 11.28. 上述构造返回的旅行回路代价不超过 $3OPT/2$。

证明. 在倍树算法的证明中，我们已经得到 $w(T) \leqslant OPT$，因此只要证明新增匹配的代价不超过 $OPT/2$，就能得到所需上界。

从一个最优旅行回路出发，保留其中属于 O 的顶点，跳过其他顶点。由三角不等式，跳过中间顶点不会增加路程，所以得到的闭合路线 Q 的代价至多为 OPT。O 中的顶点数为偶数，因此沿着 Q，可以把第 $1, 3, 5, \ldots$ 条边分为一组，第 $2, 4, 6, \ldots$ 条边分为另一组。每个顶点在两组中都恰好连着一条边，两组就分别构成完美匹配 M_1, M_2，如图 11.6 所示。这两个匹配的总代价为 $w(Q)$，所以至少一个的代价不超过 $w(Q)/2$。而 M 是所有完美匹配中代价最小的，因此

$$w(M) \leqslant \min\{w(M_1), w(M_2)\} \leqslant \frac{w(Q)}{2} \leqslant \frac{OPT}{2}.$$

加边后的图中，每条边在欧拉回路中恰好出现一次，所以这条回路的代价是 $w(T) + w(M) \leqslant 3OPT/2$。再跳过重复访问的顶点，路程只会缩短，得到的旅行回路也就不超过 $3OPT/2$。 □

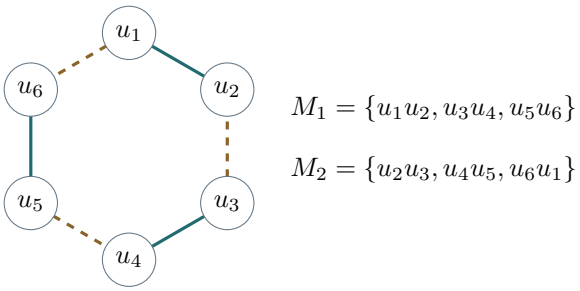


图 11.6 偶数个顶点的回路可交替分成两个完美匹配，实线和虚线分别表示两组匹配的边。

欧氏 TSP 的近似方案

前面的 3/2 保证适用于所有满足三角不等式的距离。如果城市是平面上的点，路程就是两点间的直线距离，还可以利用它们的几何位置做得更好。Arora（1998）给出了固定维数欧氏 TSP 的 PTAS，Mitchell（1999）也独立给出了平面情形的方案。

以平面情形为例，可以把平面逐层划分成较小的区域，再用动态规划组合各区域内的路线。困难在于，路线可能在区域边界上的任意位置穿过，无法逐一枚举。为此，在边界上指定有限个允许穿过的位置，并限制每段边界被穿过的次数。这样，动态规划只需记录路线从哪些位置进出，以及这些位置在区域内部如何连接，就能逐层合并结果。

这些限制会迫使路线绕行，因此算法还需要一个关键结论：适当选择划分和允许穿过的位置后，可以把最优路线调整成满足限制的路线，而总长度最多增加 εOPT。动态规划在这些受限路线中求最优解，便能得到 $(1+\varepsilon)$-近似。对每个固定的 ε，所需时间为输入长度的多项式，所以这构成 PTAS。完整算法还需给出划分方法、状态数量及绕行长度的证明，详见参考资料中的原论文。

11.9　习题

习题 11.1 (调整列表调度的输入顺序)．一套调度程序依次读入作业，每次都把当前作业交给总负载最小的机器。共有 m 台相同机器，作业的处理时间均为正数，负载相同时选择编号最小的机器。

现在已经知道这批作业的一个最优安排，但程序本身不能修改，只能调整作业的输入顺序。是否总能找到一种顺序，使这套程序也得到最优的总完成时间？若能，请根据已知的最优安排说明如何构造这一顺序；若不能，请给出反例。

习题 11.2 (连续装箱)．一排物品的大小依次为 $s_1, \dots, s_n$，其中 $0 < s_i \leqslant 1$。每个箱子的容量为 1，并且同一箱中的物品必须在原序列中连续。例如，第 2 件和第 4 件若放在同一箱，第 3 件也必须放进去。目标仍是用尽可能少的箱子装下全部物品。

1. 按原顺序使用 NF，能否总得到最优解？请证明或给出反例。
2. 按原顺序直接使用普通的 FF 或 BF，是否总能得到满足连续性要求的装法？请说明理由。

习题 11.3 (背包近似算法变体)．考虑下面这个背包算法。先按单位重量价值从大到小依次装入物品，遇到第一件放不下的物品 j 时停止。比较已经装入的物品集合 $P = \{1, \dots, j-1\}$ 与单独选择第 j 件物品的总价值，返回较好的方案。假设每件物品都能单独装入背包，价值均非负。

1. 这一算法是否仍为 2-近似？
2. 它是否总能找到最优解？能否有一个与输入无关、严格好于 2 的近似保证？

习题 11.4 (物品价值大小的影响)．0-1 背包有 n 件物品，每件价值是 $[0, n^2]$ 内的整数，重量和容量仍以任意长度的二进制数给出。能否在多项式时间内精确求解？

习题 11.5 (允许略微超重的背包)．正文中的背包近似算法严格遵守容量限制，允许总价值比最优值稍低。现在反过来，要求价值至少达到原问题的最优值，但允许总重量稍微超过容量。

给定 n 件物品，重量 w_i 为正整数，价值 v_i 为非负整数，背包容量为正整数 W。记总重量不超过 W 时的最优价值为 OPT。对于给定的 $0 < \varepsilon < 1$，设计一个设计一个运行时间关于输入长度和 $1/\varepsilon$ 都为多项式的算法，选出物品集合 S，使 $\sum_{i \in S} w_i \leqslant (1+\varepsilon)W$，$\sum_{i \in S} v_i \geqslant OPT$。

习题 11.6 (顶点覆盖的近似算法)**.** 给定无自环的无向图 $G=(V,E)$，我们希望选出尽可能少的顶点，使每条边至少有一个端点被选中。考虑下面的算法：任取一条尚未覆盖的边，把它的两个端点都选上，这样所有与这两个顶点相连的边也就被覆盖了。重复这个过程，直到全部边都被覆盖。

1. 证明这个算法能够得到一个顶点覆盖，并且选出的顶点数不超过最优值的两倍。
2. 举一个例子，说明算法选出的顶点数可以恰好为最优值的两倍。

习题 11.7 (归约不保持近似比)**.** 记图 G 的最小顶点覆盖大小为 $\tau(G)$，最大独立集大小为 $\alpha(G)$。顶点覆盖的补集是独立集，而且 $\alpha(G)=|V|-\tau(G)$，所以最优顶点覆盖的补集一定是最大独立集。那么，先求一个 2-近似顶点覆盖，再取补集，是否也能保证得到常数近似的独立集?

习题 11.8 (选择代表样本)**.** 实验室有 $n\geqslant 1$ 个样本，逐个研究的成本太高，希望从中选出尽可能少的代表。已知任意两个样本间的距离，距离满足度量的条件。给定阈值 $r>0$，要求每个样本到至少一个代表的距离不超过 r，代表也必须从这 n 个样本中选取。设计一个多项式时间算法，使选出的代表数至多为最优值的 H_n 倍，并证明这一保证。

11.10 参考答案

习题 *11.1.* 可以。记最优总完成时间为 T^*。我们从已知的最优安排出发，一边模拟程序分配作业，一边确定输入顺序。程序已经分配的作业不再改动，其余作业在最优安排中的位置可以调整，但始终保证每台机器的全部作业总时间不超过 T^*。开始时，程序还没有分配任何作业。每一步，先根据当前负载和编号确定程序将选择的机器 a，再决定下一项输入：

1. 如果当前安排中，a 还有未输入的作业，就任选其中一项作为下一项输入。程序会把它分给 a，与当前安排一致。
2. 如果 a 的作业都已输入，就从另一台机器 b 取一项未输入的作业，将它在安排中的位置改为 a，再把它作为下一项输入。程序仍会把它分给 a。

只需证明第二种调整不会使任何机器的总时间超过 T^*。设程序已分给 a,b 的作业总时间分别为 ℓ_a,ℓ_b，移动的作业处理时间为 p。因为 a 当前负载最小，$\ell_a\leqslant\ell_b$。而当前安排中，b 除了已分配的作业外，还包含这项作业，所以 $\ell_b+p\leqslant T^*$。a 已没有其他未输入的作业，移动后它的全部作业总时间就是 $\ell_a+p\leqslant\ell_b+p\leqslant T^*$。$b$ 少了一项作业，总时间只会减少。因此，上述要求始终保持。每一步恰好确定一项作业的位置，全部作业输入后，程序得到的安排仍在 T^* 内完成，因而也是最优的。 □

习题 *11.2.* 1. 能。取任意一种满足要求的装法，按物品顺序给箱子编号。记 NF 和这一装法的前 j 箱分别装到了第 g_j、q_j 件物品。下面归纳证明 $g_j\geqslant q_j$，即使用相同数量的箱子，NF 至少能装完对方已经装完的物品。第一箱的结论由 NF 的选择直接成立。假设 $g_{j-1}\geqslant q_{j-1}$。如果 $g_{j-1}\geqslant q_j$，NF 在上一箱就已经装到了第 q_j 件。否则，第 $g_{j-1}+1$ 件到第 q_j 件都是对方第 j 箱中的物品，总大小不超过 1，NF 的第 j 箱至少能装下它们。因此仍有 $g_j\geqslant q_j$。

2. 不能。对序列 $0.7,0.6,0.3$，FF 和 BF 都会把 $1,3$ 放在第一箱，2 放在第二箱，违反连续性要求。

□

习题 *11.3.* 是 2-近似，直接改写正文证明即可。它不一定最优，也不能把固定界改成小于 2。仍取容量 4、三件物品 $(w,v)=(2+\eta,2+2\eta),(2,2),(2,2)$，$0<\eta<1/2$。算法返回第一件，价值 $2+2\eta$；最优选后两件，价值 4。当 η 趋于零时，比值趋于 2。 □

习题 *11.4.* 可精确求解：使用动态规划 11.3 可以得到 $O(n^4)$ 的算法。 □

习题 *11.5.* 这次缩放重量，并向下取整。令 $\delta = \frac{\varepsilon W}{n}$，$\widehat{w}_i = \left\lfloor \frac{w_i}{\delta} \right\rfloor$，$C = \left\lfloor \frac{W}{\delta} \right\rfloor = \left\lfloor \frac{n}{\varepsilon} \right\rfloor$。以 $\widehat{w}_i$ 为重量、C 为容量，保持价值不变，使用第八章的动态规划求解这个新的 0-1 背包问题，时间复杂度显然符合 FPTAS 要求。设原问题的最优物品集合为 S^*。向下取整后，$\sum_{i \in S^*} \widehat{w}_i \leqslant \left\lfloor \frac{\sum_{i \in S^*} w_i}{\delta} \right\rfloor \leqslant C$，所以 S^* 在新问题中仍然可行。算法求出了新问题的最优解，价值自然不低于 OPT。另一方面，每件物品的真实重量都小于 $\delta \widehat{w}_i + \delta$。算法至多选择 n 件物品，把每件物品取整时少算的部分加回来，总重量增加不到 $n\delta$，所以输出集合满足 $\sum_{i \in S} w_i \leqslant \delta \sum_{i \in S} \widehat{w}_i + n\delta \leqslant \delta C + \varepsilon W \leqslant (1+\varepsilon)W$。 □

习题 *11.6.*

1. 每次选择的边此前尚未覆盖，选中它的两个端点后，至少会多覆盖这一条边。因此，算法最终会覆盖全部边，得到一个顶点覆盖。设算法一共取出了 t 条边。每条新取出的边都没有端点被选中，因此它与此前取出的边没有公共端点。任何顶点覆盖都必须为这 t 条边分别选出至少一个端点，而同一个顶点不能同时覆盖其中两条边，所以 $OPT \geqslant t$。算法为每条边选中了两个端点，共选出 $2t \leqslant 2OPT$ 个顶点，因此具有 2-近似保证。
2. 图中只有一条边时，算法选择它的两个端点，而最优方案只需选择一个，比值恰好为 2。

□

习题 *11.7.* 取 G 为 $m \geqslant 1$ 条互不相交的边。最小顶点覆盖大小为 m，最大独立集大小也为 m。习题 11.6 中的算法会选中全部 $2m$ 个端点，是合法的 2-近似覆盖。但取补集得到空集，独立集价值为 0，与正的最优值相比没有任何有限近似比。 □

习题 *11.8.* 对每个样本 p，建立集合 $S_p = \{q : d(p,q) \leqslant r\}$。选择 p 为代表，就能满足 S_p 中所有样本的要求。因此，以全部样本为元素、以各个 S_p 为可选集合，并令每个集合的代价为 1，就得到一个等价的集合覆盖问题，此后细节不再赘述。 □

11.11 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十一讲。
2. 金鱼马的知乎专栏。本讲基本概念与装箱问题的讲解参考了其中的文章，感谢金鱼马对讲义的支持，也推荐对优化问题感兴趣的读者阅读。
3. Mark Allen Weiss，*Data Structures and Algorithm Analysis in C*，第二版，第 10.1.3 节介绍装箱问题及 NF、FF、FFD 算法。
4. Jon Kleinberg、Éva Tardos，*Algorithm Design*，第 11.1–11.3 节介绍调度、中心选择与集合覆盖，第 11.8 节介绍背包的近似方案。
5. Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein，*Introduction to Algorithms*，第三版，第 35.1–35.2 节介绍顶点覆盖、度量 TSP 与一般 TSP 的不可近似性。
6. David P. Williamson、David B. Shmoys，*The Design of Approximation Algorithms*，2011。作者公开版第 1.1 节定义近似保证，第 3.1 节介绍近似方案。
7. Pierluigi Crescenzi、Viggo Kann，*A Compendium of NP Optimization Problems*。近似算法与问题分类介绍 NPO、APX、PTAS 和 FPTAS。

8. *CO 454: Scheduling Theory*，2009 年春，第 12 讲介绍相同并行机器上的列表调度与 LPT 分析。

9. David S. Johnson，*Fast Algorithms for Bin Packing*，*Journal of Computer and System Sciences*，8(3):272–314，1974。介绍 Fit 类装箱算法及其渐近分析。

10. György Dósa、Jiří Sgall，*First Fit Bin Packing: A Tight Analysis*，STACS 2013，538–549；*Optimal Analysis of Best Fit Bin Packing*，ICALP 2014，429–441。分别证明 FF、BF 的绝对近似比为 1.7。

11. György Dósa，*The Tight Bound of First Fit Decreasing Bin-Packing Algorithm Is* $FFD(I) \leqslant 11/9\,OPT(I)+6/9$，ESCAPE 2007，1–11；György Dósa、Rongheng Li、Xin Han、Zsolt Tuza，*Tight Absolute Bound for First Fit Decreasing Bin-Packing*，*Theoretical Computer Science*，510:13–61，2013。讨论 FFD 的紧界；本讲所用紧例也见 Dósa 的研究文稿第 3 章。

12. János Balogh、József Békési、György Dósa、Jiří Sgall、Rob van Stee，*The Optimal Absolute Ratio for Online Bin Packing*，SODA 2015，1425–1438。给出绝对竞争比为 $5/3$ 的在线装箱算法。

13. Nicos Christofides，*Worst-Case Analysis of a New Heuristic for the Travelling Salesman Problem*，CMU Management Sciences Research Report 388，1976。提出度量 TSP 的 $3/2$-近似算法。

14. Sanjeev Arora，*Polynomial Time Approximation Schemes for Euclidean Traveling Salesman and Other Geometric Problems*，*Journal of the ACM*，45(5):753–782，1998；Joseph S. B. Mitchell，*Guillotine Subdivisions Approximate Polygonal Subdivisions: A Simple Polynomial-Time Approximation Scheme for Geometric TSP, k-MST, and Related Problems*，*SIAM Journal on Computing*，28(4):1298–1309，1999。分别介绍固定维数和平面欧氏 TSP 的近似方案，也可阅读 Arora 的几何优化综述第 1 节。